Naive algorithm for Pattern Searching

Last Updated : 31 Mar, 2026

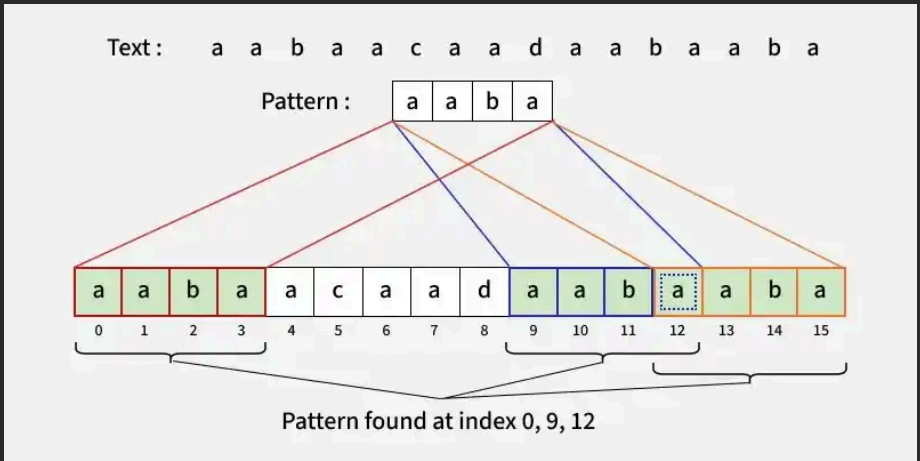
Given a **text** string of length **n** and a **pattern** of length **m**, find all starting indices where the pattern occurs in the text.

Note: You may assume that n>m.

Examples:

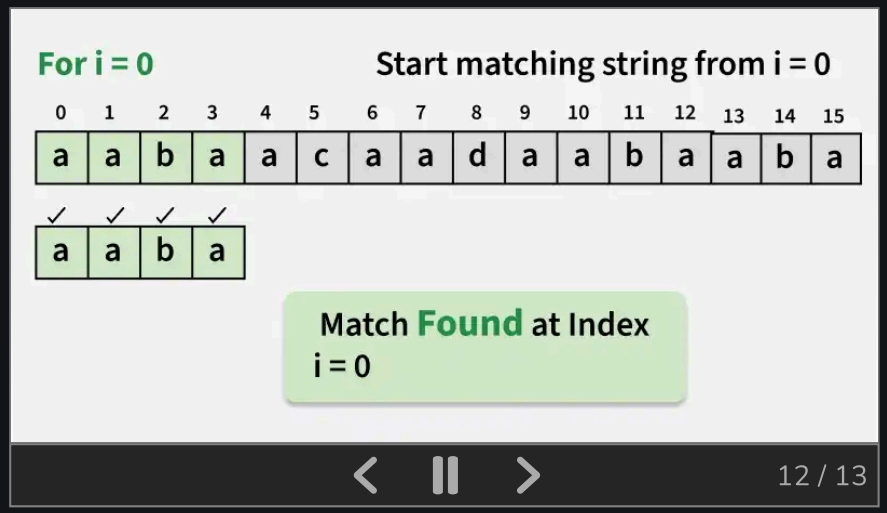
Input: *text = "geeksforgeeks", pattern = "geeks"*
Output: *[0, 8]*
Explanation: *The string "geeks" occurs at index 0 and 8 in text.*

Input: *text = "aabaacaadaabaaba", pattern = "aaba"*
Output: *[0, 9, 12]*



Naive Pattern Searching Algorithm

The pattern moves over the text one position at a time and characters are compared. If all characters match, the index is stored; otherwise, the next position is checked.



C++ Java Python C# JavaScript

```cpp
#include <vector>
using namespace std;

// Function to search for all occurrences of
'pat' in 'txt' using Naive Approach
vector<int> search(const string &pat, const
string &txt)
{
    // Length of the pattern
    int m = pat.length();

    // Length of the text
```

```cpp
    int n = txt.length();

    vector<int> ans;

    // Slide the pattern over text one by one
    for (int i = 0; i <= n - m; i++)
    {
        int j;

        // Check for pattern match at index i
        for (j = 0; j < m; j++)
        {
            if (txt[i + j] != pat[j])
                break;
        }

        // If pattern matches, store index
        if (j == m)
            ans.push_back(i);
    }

    return ans;
}

int main()
{
    string txt = "aabaacaadaabaaba";
    string pat = "aaba";

    vector<int> res = search(pat, txt);

    for (auto it : res)
    {
        cout << it << " ";
```

```
    }

    return 0;
}
```

Output

```
0 9 12
```

Best Case Scenario

The best case occurs when the first character of the pattern does not match any character in the text.

Example: txt = "AABCCAADDEE" , pat = "FAA" . Only one comparison is done at each position. Time Complexity: O(n)

Worst Case Scenario

The worst case occurs when many characters match repeatedly.

Case 1: All characters are the same. Example: txt = "AAAAAAAAAAAAAAAAA" , pat = "AAAAA"

Case 2: Only the last character is different. Example: txt = "AAAAAAAAAAAAAB", pat = "AAAAB"

In these cases, most characters match at every position, leading to repeated comparisons. Time Complexity: O(m × (n

- m + 1))

Time Complexity

O(n × m), because every possible starting position in the text is checked and up to m characters are compared at each position.

Auxiliary Space

O(1), only a few variables are used, and no extra data structures are required.

Although strings which have repeated characters are not likely to appear in English text, they may well occur in other applications (for example, in binary texts). The KMP matching algorithm improves the worst case to O(n). We will be covering KMP in the next post. Also, we will be writing more posts to cover all pattern searching algorithms and data structures.

Related Articles

- KMP Algorithm for Pattern Searching
- Rabin-Karp Algorithm for Pattern Searching
- Z Algorithm for Pattern Searching